

Dictionaries in Python

Consider the example:

```
Romania-transition-model = {  
    'Arad': [('Zerind', 75), ('Timisoara', 118), ('Sibiu', 140)],  
    'Zerind': [...],  
    ...  
}
```

Key
of the dict.

We can imagine it as an Associative array, that has keys as cities and its elements are lists of tuples.

- So to get the main city we need to access the key
- to get the inside cities: `Amay[city][0]`
(Zerind, 75)

to get only Zerind.

[to get a city] → `Amay[city][0][0]`
first element of the tuple

→ In order to get the whole list using functions of Python:

```
transition-model.get(city);  
    ↑  
name of the key.
```


Difference between the set and explored list

- List: maintains an order / allow duplicate elements
- Set: Does not have an order / does not allow duplicates

Now in our Search algorithm,

- Explored is a set because we ^{don't} care about the order
- Frontier priority queue is a list because the order matters!

Next Search Problems:

Labyrinth.

Some functions

def expand_node(self, node):

at first you get all possible actions → children = []

possible = self.get_possible_actions(node),

for action in possible:

create a copy of the current node and apply to it

the action wanted.

node_copy = ~~node~~ deepcopy(node)
Copy.

node_copy.state = self.apply_action(action[0])

children.append(node_copy)

if isinstance(action, tuple):
action = action[0]
(city, cost)

To get both the Index and the element in the loop
We have to use the function Enumerate.

★ example:

```
def find-first-empty(self, node):  
    for i, row in enumerate(node.state):  
        for j, element in enumerate(row):  
            if element == 0:  
                return (i, j)  
    return None // nothing found.
```

★ How loops work in Python:

`for i in range(a, b):` ← the interval
We'll get write:
 { a inclusive
 b exclusive
 }
[a, b[

★ When we use a loop and we don't use
the `i` of the loop in Python

■ We can write the loop like:

`goal_state = [ [! for _ in range(5)] for _ in range(5)]`

↓ we don't care about it!!

The whole problem was
Popping that element from
the frontier !!!!!

in the A* →

remove from the frontier
that element !!!!!

When writing the "A*" Case: in an
graph search.

♦ We always need to pop from the frontier !!!

if type = "A*":

min-index = min(range(len(frontier)), key = lambda
index: problem.heuristic(frontier[index])
+ frontier[index].cost)

VERY
CRUCIAL!!!
mode = frontier.pop(min-index)

Explaining the line for A*:

♦ min(): built in function to get smallest item in an iterable this
♦ Key: is the parameter that will return the Key for
each element in the
iterable.

♦ Lambda x: problem.heuristic(x) + x.cost returns the sum.

the sum.
returns the ~~cost~~
value for the cost + h